Binary Search is a fast search algorithm that works on sorted arrays.
It repeatedly halves the search interval.

How does it work?
1. Start with the whole array.
2. If the target value is equal to the middle element, return the index.
3. If the target value is less than the middle element, narrow the search to the lower half.
4. If the target value is greater than the middle element, narrow the search to the upper half.
5. Repeat until found or the interval is empty.

Let's walk through the binarySearch function **line by line**. Assuming the length of the array is 5, and the target to search is 4:

- Array: [1, 2, 3, 4, 5]
- Target: 4

Array indices:

- Index 0 → 1
- Index 1 → 2
- Index 2 → 3
- Index 3 → 4
- Index 4 → 5

Here is the PHP function for reference:

```
function binarySearch($arr, $target) {
 $low = 0;
 $high = count($arr) - 1;
 while ($low <= $high) {
  $mid = $low + (int)(($high - $low) / 2);
  if ($arr[$mid] == $target)
   return $mid;
  if ($arr[$mid] < $target)
   $low = $mid + 1;
  else
   $high = $mid - 1;
 }
 return -1;
}
```

Function start and initial setup
function binarySearch($arr, $target) {

- Defines a function named binarySearch.
- $arr is [1, 2, 3, 4, 5].
- $target is 4.

$low = 0;

- Start searching from the **leftmost** index.

Current state:

- $low = 0

$high = count($arr) - 1;

- count($arr) is 5 because there are 5 elements.
- So $high = 5 - 1 = 4.

Current search range (by index):

- $low = 0, $high = 4 → looking at the whole array [1, 2, 3, 4, 5].

First loop iteration

while ($low <= $high) {

- Check: 0 <= 4 → true, so enter the loop.

$mid = $low + (int)(($high - $low) / 2);

Compute step by step:

- $high - $low = 4 - 0 = 4
- 4 / 2 = 2
- (int)2 is 2
- $mid = 0 + 2 = 2

So:

- $mid = 2
- $arr[2] is 3.

if ($arr[$mid] == $target)

- Check: 3 == 4 → false.
- So we do not return yet.

if ($arr[$mid] < $target)

- Check: 3 < 4 → true.

Because this is true:

$low = $mid + 1;

- $low = 2 + 1 = 3.

Now:

- $low = 3, $high = 4.

Interpretation:

- Middle value (3) is less than 4.
- So the target must be in the **right half**: indices 3 to 4 (values [4, 5]).

Loop goes back to the top.

Second loop iteration

while ($low <= $high)

- Check: 3 <= 4 → true, keep going.

$mid = $low + (int)(($high - $low) / 2);

Now:

- $low = 3, $high = 4
- $high - $low = 4 - 3 = 1

- 1 / 2 = 0.5
- (int)(0.5) → 0
- $mid = 3 + 0 = 3

So:

- $mid = 3
- $arr[3] is 4.

if ($arr[$mid] == $target)

- Check: 4 == 4 → true.

So:

return $mid;

- The function returns 3.

This means:

- The target 4 is found at **index 3** in the array.

The function stops here; the rest of the code (the second if and the return -1) is never reached in this case.

So with [1, 2, 3, 4, 5] and target 4, the function:

1. Looks at middle index 2 (value 3), decides to move right.
2. Narrows range to indices 3–4.
3. Looks at middle index 3 (value 4), finds the target, and returns 3.

To check your understanding: if the same sorted array is [1, 2, 3, 4, 5] but the target is 5, what will the sequence of (low, high, mid) values be on each loop iteration before it returns?

Final Task Number 1:

Write a complete Binary Search PHP code that allows the user to enter the length of the array and the value of each element. If the array is not sorted, it must be sorted first before the binary search takes place.

Sample Output 1, if the target is found:

```
Enter the length of the array: 5
Enter 5 elements (any order):
Element 1: 5
Element 2: 3
Element 3: 4
Element 4: 1
Element 5: 2

Original array: [5, 3, 4, 1, 2]
Sorted array: [1, 2, 3, 4, 5]

Enter the value to search: 4

✅ Element 4 found at index: 3
```

Sample Output 2, if the target is not found:

```
Enter the length of the array: 5
Enter 5 elements (any order):
Element 1: 5
Element 2: 3
Element 3: 4
Element 4: 1
Element 5: 2

Original array: [5, 3, 4, 1, 2]
Sorted array: [1, 2, 3, 4, 5]

Enter the value to search: 6

❌ Element 6 not found in the array
```